ProGineer Technologies
WE DELIVER

TECHNICAL TASK ASSIGNMENT

Pagination in ASP.NET Core Web APIs

Backend Development Training — Weekly Practical Task

TARGET PROJECT	DOMAIN	DELIVERABLE
Donation Management API	**ASP.NET Core & EF Core**	**Research & Architectural Report**

00 Task Overview

In this task, you will research the three primary pagination strategies used in modern RESTful APIs and determine which approach is most suitable for our enterprise **Donation Management API**. API performance, data integrity under high concurrency, and database efficiency are critical factors for this platform.

01 Part 1: Research & Comparison

Conduct an in-depth technical analysis comparing the following three pagination strategies:

1. **Offset-Based Pagination** (e.g., `OFFSET 40 FETCH NEXT 20 ROWS ONLY`)
2. **Cursor-Based / Keyset Pagination** (e.g., `WHERE Id > @LastId ORDER BY Id ASC`)
3. **Page-Based Pagination with Total Count** (Page number + page size + total record count)

For each approach, thoroughly document and evaluate:

- **Mechanism & Execution:** How it operates at the HTTP API level and translates into SQL queries generated by Entity Framework Core.
- **Advantages & Disadvantages:** Developer UX, front-end client integration complexity, support for random page access, and data consistency under active mutations.
- **Performance & Scalability:** Database index performance, memory overhead, and query latency trends as dataset sizes scale to millions of records.
- **Use Case Suitability:** Specific architectural scenarios where the approach excels or degrades.

> **KEY EVALUATION FOCUS**
>
> Pay specific attention to how each pagination pattern handles real-time dataset changes (e.g., new donations inserted while a user navigates between pages) and SQL query plan performance on high-volume tables.

02 Part 2: Choose the Best Approach

Select the single pagination strategy that best aligns with the operational requirements of the **Donation Management API**. Justify your choice by addressing the following technical questions:

- **Architectural Justification:** Why is this approach optimal for our specific API and data access patterns compared to alternatives?
- **Trade-Off Analysis:** What inherent limitations does your selected approach introduce, and how will you mitigate them?
- **Alternative Scenarios:** Under what circumstances or requirements would you pivot and recommend a different pagination mechanism?

03 Part 3: Implementation Plan

Provide a detailed technical blueprint for implementing your chosen strategy using **ASP.NET Core Web API** and **Entity Framework Core**. Your explanation should include:

Component	Implementation Details
API & Request DTO	Endpoint definition, HTTP query parameters, and validation rules.
EF Core LINQ Construction	Dynamic query construction, index leverage, and avoiding N+1 queries.
Response Structure	Paginated wrapper DTO (items, pagination metadata/pointers, navigation flags).
Sorting & Filtering	Handling dynamic sorting and filters (e.g., date ranges, minimum amount).
Deterministic Order	Ensuring predictable pagination with secondary tie-breaker columns (e.g., Primary Key).

04 Submission Requirements

- **Documentation:** Submit a structured report covering all research, trade-off comparisons, and implementation plans.
- **Code Snippets:** Include production-ready ASP.NET Core Controller and EF Core LINQ snippets demonstrating your implementation.
- **Technical Defense:** Be prepared to explain and defend your architectural recommendations during the peer engineering review.

> **IMPORTANT NOTICE**
>
> There is no pre-determined "correct" pagination type. Your recommendation must be supported by empirical technical trade-offs based on database performance, query execution plans, and client requirements.